

Preprocessore

Preprocessore

Le direttive del preprocessore in C++, come

Cos'è il preprocessore?

Quando scrivi un programma in C++, prima ancora che il compilatore legga il tuo codice, un altro programma — chiamato **preprocessore** — fa una "pulizia e preparazione" del testo.

Immagina di consegnare un compito scritto a matita a un assistente: lui prima cancella i riferimenti, sostituisce alcune parole con quelle giuste, aggiunge i fogli mancanti... e solo dopo lo consegna al professore (il compilatore) per la correzione.

Le istruzioni che dai al preprocessore si chiamano **direttive** e iniziano sempre con il simbolo `#`.

`#include` — Aggiungere altri file

`#include` dice al preprocessore: "Copia il contenuto di questo file e mettilo qui."

È come dire: "Aggiungi questo capitolo del libro al mio testo."

Librerie del sistema

Per includere le librerie già pronte che vengono con C++, si usano le parentesi angolari `<>`:

```
#include <iostream>    // per stampare e leggere dati
#include <string>       // per usare le stringhe
#include <cmath>        // per le funzioni matematiche
#include <vector>       // per usare i vettori
```

Il compilatore sa già dove trovare questi file nel tuo computer.

File tuoi

Per includere file che hai scritto tu, si usano le virgolette `""` :

```
#include "studente.h"          // file nella stessa cartella
#include "modelli/persona.h"    // file in una sottocartella
```

Il compilatore cerca prima nella cartella del tuo progetto, poi nelle cartelle standard.

#define — Creare una sostituzione automatica

`#define` è come creare una scorciatoia: dici al preprocessore *"ogni volta che vedi questa parola, sostituiscila con quest'altra"*.

Costanti con `#define`

```
#define PI 3.14159265358979
#define MAX_STUDENTI 30
#define SALUTO "Ciao, mondo!"

int main() {
    // Il preprocessore sostituisce PI prima ancora di compilare:
    // double area = 3.14159265358979 * 5.0 * 5.0;
    double area = PI * 5.0 * 5.0;
    return 0;
}
```

Oggi si preferisce usare `const` al posto di `#define` per le costanti, perché è più sicuro. Ma `#define` si vede spesso nel codice esistente.

Macro con parametri (scorciatoie con calcoli)

Puoi creare scorciatoie che accettano valori:

```
#define MASSIMO(a, b) ((a) > (b) ? (a) : (b))
#define QUADRATO(x) ((x) * (x))

int main() {
    int x = 5, y = 3;
    cout << MASSIMO(x, y) << endl;    // stampa 5
    cout << QUADRATO(4) << endl;      // stampa 16
    return 0;
}
```

Attenzione: queste "macro-funzioni" possono dare risultati strani. Preferisci le funzioni normali:

```
// PROBLEMA: n viene incrementato due volte!
int n = 5;
int r = QUADRATO(n++);    // diventa: ((n++) * (n++)) – comportamento
                           imprevedibile

// SOLUZIONE: usa una funzione normale
int quadrato(int x) { return x * x; }
```

#undef – Cancellare una scorciatoia

```
#define VALORE 42
cout << VALORE << endl;    // 42

#undef VALORE
// Da qui in poi VALORE non esiste più
// cout << VALORE << endl;    // ERRORE!
```

Compilazione condizionale — Includere codice solo in certi casi

Puoi dire al preprocessore:

"includi questa parte di codice solo se questa condizione è vera"

.

È come avere due versioni di un documento e scegliere quale stampare:

```
#define DEBUG    // commenta questa riga per disabilitare i messaggi di
debug

int main() {
    int x = 42;

#ifdef DEBUG
    // Questa riga viene inclusa SOLO se DEBUG è definito
    cout << "Debug: x = " << x << endl;
#endif

    cout << "Valore: " << x << endl;
    return 0;
}
```

- `#ifdef NOME` = "se NOME è definito, includi il codice che segue"
- `#ifndef NOME` = "se NOME NON è definito, includi il codice che segue"
- `#endif` = "fine del blocco condizionale"

Include guard — Evitare di includere lo stesso file due volte

Immagina di copiare e incollare lo stesso capitolo due volte in un libro: sarebbe un problema. Stessa cosa in C++.

Quando scrivi un file `.h`, usa sempre questa protezione:

```
// studente.h
#ifndef STUDENTE_H    // se STUDENTE_H non è ancora definito...
#define STUDENTE_H    // ...definiscilo (ed esegui il codice qui sotto)

// Tutto il contenuto del file va qui
void salutaStudiante();
struct Studente { int eta; };

#endif    // fine della protezione
```

La seconda volta che questo file viene incluso (magari attraverso un altro file), `STUDENTE_H` è già definito, quindi tutto il contenuto viene saltato automaticamente.

In alternativa, puoi usare `#pragma once`, che fa la stessa cosa in modo più semplice:

```
// studente.h
#pragma once    // includi questo file al massimo una volta

void salutaStudiante();
```

`#pragma once` è supportato da quasi tutti i compilatori moderni, anche se non è parte dello standard ufficiale.

#error — Generare un errore personalizzato

Puoi far fallire la compilazione con un messaggio tuo:

```
#ifndef MIA_LIBRERIA_H
#error "Devi includere mia_libreria.h prima di questo file!"
#endif
```

Se la condizione si verifica, il compilatore si ferma e mostra il tuo messaggio.

Macro predefinite — Informazioni automatiche

Il preprocessore definisce alcune parole speciali in automatico:

```
cout << __FILE__ << endl;    // nome del file corrente
cout << __LINE__ << endl;    // numero della riga corrente
cout << __DATE__ << endl;    // data in cui è stato compilato il programma
cout << __TIME__ << endl;    // ora in cui è stato compilato
```

Sono molto utili per il debug — per sapere esattamente dove è successo un problema:

```
#ifdef DEBUG
// Crea un messaggio di log con il nome del file e il numero di riga
#define LOG(msg) cout << "[" << __FILE__ << ":" << __LINE__ << "]" " << msg
<< endl
#else
#define LOG(msg)    // in modalità normale, LOG non fa nulla
#endif

int main() {
    int x = 42;
    LOG("Il valore di x è: " << x);    // messaggio visibile solo in
    modalità debug
    return 0;
}
```